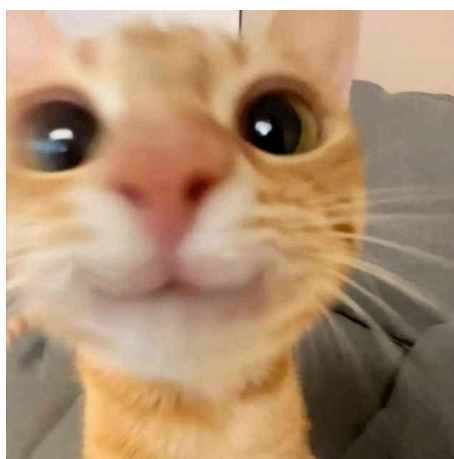


Принципы работы антиплагиата для кода

1. Предисловие (обязательно к прочтению!)

Я поняла, что (пока) не смогу разобраться в теме, которую взяла изначально в прошлом полугодии, поэтому видоизменила ее, немного упростив себе задачу - так я действительно пойму что-то из того, в чём пытаюсь копаться. Теперь я хочу разобрать некоторые принципы работы антиплагиата, который находит совпадения между кодами из разных источников (например, между кодами от разных участников конкурса).

Даже если от этого работа покажется сильно проще (и вообще, кто в 2026 году крадет код у товарищей :)), даже если сама работа окажется короче, чем предисловие (я пока не начала писать саму статью), даже если язык, которым она будет написана, будет простым, и работа будет корявой, как минимум для меня в ней будет больше смысла - так же, как и с прошлой темой, у меня сейчас нет полного понимания того, как это работает, поэтому новый для себя материал я изучу, и причем этого материала в доступе больше, он проще для понимания (не настолько, чтобы работа стала бессмысленной, но настолько, чтобы мне не казалось невозможным сделать эту работу без ИИ, который почти (а может и не почти) полностью лишает ее смысла). Работа сделана моими руками, в том числе перевод в человекочитаемый формат. Я заявляю, что работа в предыдущем семестре была сделана с нуля за пару часов до дедлайна, и 100% ее источников составляет ИИ, поэтому я её даже не считаю за работу. Сейчас я встаю на путь истины, очищаю свою ауру (или хотя бы не порчу ее еще больше). Спасибо.



upd: Я дописала работу. Я действительно разобралась в том, что написала. Кажется, цель достигнута, хоть работа правда очень небольшая.

2. Введение

Итак, очевидно, что в лоб сравнивать коды как текст (например, с помощью расстояния Левенштейна) крайне неэффективно - мы как минимум получим false negative результат,

если один человек возьмет код своего друга и изменит в нем названия переменных, что делает такой антиплагиат крайне неэффективным - работает только в том случае, если у студентов только три клавиши на клавиатуре (догадаться, какие это клавиши, оставим читателю в качестве упражнения). Значит, нужно уметь сравнивать логику или структуру кодов.

Не мешало бы сделать оговорку - конечно, может встретиться и false positive ошибка в задачах с коротким кодом или примитивным решением, но абсолютно точный антиплагиат сделать скорее невозможно в целом, да и нужда в антиплагиате возникает скорее тогда, когда люди решают более замысловатые задачи (не A-B div4 codeforces).

3. Подготовка кода

Методов проверки исходных кодов на антиплагиат по логике и структуре хоть отбавляй, но их объединяет одно - сначала нужно отформатировать код и убрать все “бантики”, которые мешают: в частности, комментарии, названия переменных (они заменяются на нейтральные вроде var1, var2 и т. п.), удаляются неиспользуемые переменные и no-operation конструкции (пустые циклы, например).

4. Проверка с помощью токенизации кодов (Token-based method)

В ходе этого метода код преобразуется в последовательность токенов. Каждый токен может представлять собой ключевое слово языка (например, int, for), оператор, числовое значение и т.п. Каждому токenu присваивается уникальное символьное или числовое обозначение, после чего строится N-грамма определенного размера - последовательность из N элементов со сдвигом на один элемент (после группы из N токенов пропускаем один токен и берем следующие N токенов). Чтобы было понятнее, приведу пример из [статьи](#) для N = 4:

Исходная последовательность токенов: K I O N K I O N...

N-граммы: [K I O N] [I O N K] [O N K I]

Далее берутся множества N-грамм от двух файлов и вычисляется отношение размера пересечения множеств к размеру их объединения. Если отношение превышает установленный порог, файлы помечаются как совпадающие. Далее для поиска совпадающего фрагмента кода используется алгоритм поиска наибольшей общей подстроки ([LCS](#)) для исходной последовательности токенов.

Чем хорош данный метод?

Как и остальные принципы, которые не основываются на проверке кодов как обычных текстов, он устойчив к “косметическим” изменениям украденного кода. Помимо этого, даже если человек поменяет местами некоторые строки кода, N-граммы, построенные на этих строках, с большой вероятностью не изменятся, поэтому скорее всего не изменится и множество N-грамм.

Чем он плох?

При изменении оператора токен сразу меняется, что с большой вероятностью изменяет набор N-грамм. Помимо этого, такое изменение токена мешает нахождению совпадающего фрагмента как LCS последовательностей токенов - одну большую подстроку можно легко разбить на две части, и в таком случае фрагмент уже не находится полностью.

Также сложность возникает при подборе такого N, чтобы возникало минимальное количество ошибок - при маленьком N пересечение множеств будет слишком большим, что даст false positive результат, а при больших (например, `N = code_as_text.size()`) алгоритм уже не будет устойчив к таким изменениям, как перестановка строк кода.

Этот метод используется в небезызвестном антиплагиате JPlag.

5. Tree-based method

Данный метод использует абстрактное синтаксическое дерево (Abstract Syntax Tree - AST). После нормализации кода по нему строится дерево: в качестве узлов используются операции, обобщенные в группы, почти без конкретики. Например, prefix increment and decrement (`++x`, `--x`), additive operations (`+`, `-`), equality operations (`==`, `!=`), `forLoopAST` (цикл `for`), `blockAST` (тело цикла). При этом опять же игнорируются операции, не влияющие на работу программы (неиспользуемые переменные), удаляется то, что присуще многому (`decl` при объявлении переменной, `identifier` - имя переменной, `constant` - число).

Связи между узлами (рёбрами) задаются исходя из логической вложенности: например, в цикле `for` связь будет построена так: От `forLoopAST` к инициализации, условию, шагу и `blockAST`, от `blockAST` к операциям, которые лежат в теле цикла. Думаю, интуитивно понятно, почему такая структура является деревом - одна и та же операция не может лежать в разных блоках, если один не вложен в другой.

Также каждому узлу соответствует [хеш](#) (fingerprint или отпечаток, как это описано в [статье](#)), углубляться в формулу подсчета я сейчас не буду. Важно лишь то, что мы можем видеть уникальность узлов с помощью хеша, а также значение хеша зависит от дочерних вершин узла, то есть мы также можем определить уникальность поддерева вершины.

Далее происходит сравнение деревьев. По одному из них (пусть это будет дерево A) запускается [BFS](#). Во время обработки вершины мы проверяем наличие узла с таким же хешем в другом дереве (обозначим его за B). Если такой узел найден, мы отсекаем поддерево этой вершины и не проверяем его, так как уже знаем, что в нём есть совпадение, увеличивая счетчик совпавших узлов на вес отсеченного поддерева.

Такой алгоритм далее запускается на дереве B для сравнения его с A. В конце каждого из двух запусков считаем отношение значения счетчика к количеству вершин в дереве, которое сравниваем с потенциальной “копией”, складываем их и проверяем, превышает ли полученное значение установленный порог.

Чем хорош этот метод?

У данного подхода довольно высокая точность. Также он работает за $O(n)$, где n - количество узлов в дереве, при этом неасимптотически оптимизируется за счет отсечений поддеревьев. И, конечно, он устойчив к заменам типов данных (`int` -> `unsigned int`), использованию эквивалентных конструкций (`for` -> `while`), перестановке блоков кода.

Какие минусы?

Добавление операторов снижает вероятность того, что потенциально скопированный кусок кода будет обнаружен, так как операции могут лежать в разных группах (те же самые `++x` и `x += 1` уже определены как разные). Помимо этого, такой метод лучше использовать именно на задачах со сложной логикой решения, так как иначе большая часть решений будет помечена как неоригинал по ошибке.

6. Как работает MOSS - один из самых популярных антиплагиатов (small bonus chapter)

Принцип работы **MOSS** (используется не только для кода, вернее, по большей части не для кода, а для письменных работ) в чём-то схож с методом, упомянутым в пункте 4 о токенизации, поэтому я вкратце про него расскажу.

Эта система антиплагиата использует скользящее окно ([sliding window](#)). Каждому символу присваивается хеш, выбирается длина окна k . Во время движения окна сохраняем минимальное значение хеша в этом окне и позицию элемента с этим хешом относительно начала текущего окна.

После такого препроцессинга можно выявлять совпадающие места в текстах (и кодах): даже если поменять местами фрагменты, целые куски внутри них будут неизменны, и мы можем легко найти такие совпадения, так как пары из хешей и позиций в окнах, которые мы сохранили на предыдущем шаге, скорее всего будут одинаковыми.

7. Материалы, которые я так или иначе использовала (даже если это комментарии в чьём-то блоге на codeforces)

<https://habr.com/ru/articles/583882/>

<https://ps-group.github.io/compiler/ast>

<https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9960266>

<https://cyberleninka.ru/article/n/primenenie-metoda-tokenizatsii-dlya-poiska-shozhih-fragmentov-koda-v-studencheskih-rabotah/viewer>
<https://theory.stanford.edu/~aiken/publications/papers/sigmod03.pdf>
<https://codeforces.com/blog/entry/65874>
<https://algorithmica.org/tg/bfs>
<https://www.geeksforgeeks.org/dsa/window-sliding-technique/>
<https://habr.com/ru/articles/534596/>